



QSOE

Quick and Secure Operating Environment

Networking Guide

Transparent distributed networking with QSP

Document: QSOE-NET

Revision: Rev A — Draft

Applies to: QSOE 0.2

Yuri Zaporozhets

August 2026

© 2026 Yuri Zaporozhets.

Licensed under CC BY-ND 4.0 — verbatim redistribution with attribution; no derivatives.

<https://creativecommons.org/licenses/by-nd/4.0/>

Contents

1	What Transparent Distributed Networking Is	1
1.1	Why this is not “networking”	1
1.2	The names	1
1.3	What it looks like	2
1.4	What TDN carries, and what it does not	2
1.5	Clean-room, and why that matters here	3
2	Stations, Call Signs, and the Net	4
2.1	Station ids are assigned by hand	4
2.2	Call signs	4
2.3	The roster, and knowing whether two copies agree	5
2.4	Configured stations and discovered ones	5
2.5	/net, the namespace of stations	5
2.6	Node descriptors	5
3	Setting Up a Net	7
3.1	What the hardware has to be	7
3.2	The interface driver	7
3.3	The roster file	7
3.4	Who <i>we</i> are: ask the board	8
3.5	Running the manager	8
3.6	Bringing up a two-station net	9
3.7	Stations beyond this switch	10
4	Using the Net	11
4.1	The path form	11
4.2	What a remote open actually does	11
4.3	Everything after the open is opaque	12
4.4	Errors, and what each one tells you	12
4.5	Addressing a station from a program	13
4.6	What the far station sees	13
4.7	Limits you can hit today	13
5	Diagnosing a Net	15
5.1	qspinfo — what this station believes	15
5.2	netpoke — is there a wire at all	16
5.3	Common failures, in the order to check them	16
6	How QSP Works	18
6.1	Three layers	18
6.2	Discovery and liveness	18
6.3	Circuits	19

6.4	Transactions	19
6.5	Priority crosses the wire	19
6.6	Traffic classes	20
6.7	Everything is per peer	20
6.8	Why the manager does not use the resource-server framework	20
6.9	The threads	20
6.10	What QSOE refuses to inherit	21
7	The State of TDN in 0.2	22
7.1	What works	22
7.2	What does not, yet	22
7.3	Sizing a system against this	23
7.4	The road	23

Chapter 1

What Transparent Distributed Networking Is

QSOE is built on message passing. A program does not call a driver, it sends a message to one; it does not link a filesystem, it opens a path that some resource server answers. Everything above the kernel is a client or a server, and the only thing between them is `MsgSend`.

Transparent Distributed Networking — **TDN** — is what happens when you let that message cross a wire.

That is the whole idea, and it is worth stating plainly before any of the machinery: *a QSOE program that works on one machine works across several without being changed, recompiled, or told that anything has changed.* If a program can read `/dev/ser1` on the machine it runs on, it can read `/net/unmatched/dev/ser1` on a machine across the room, and the code that does so is the same `open` and the same `read`. The remote server does not know it is being reached remotely. The client does not know its bytes went through a cable. Nobody was ported.

1.1 Why this is not “networking”

Most operating systems, asked to connect two machines, hand you a socket. A socket is a byte pipe with an address on each end, and everything that makes distributed software *work* — naming, request and response, error reporting, access control, ordering, flow control — is left for you to build on top of it, again, in every program.

QSOE already has all of those things. Its message passing is request/response by construction, it has a pathname space for naming, it has POSIX `errno` values for failure, it has `uid` and `gid` for access. TDN does not invent a second, parallel set of them for use between machines. It extends the ones already there.

So there is no socket API in this guide, no listening port, no protocol to design, and — deliberately — **no IP**. QSP, the protocol behind TDN, rides directly on Ethernet. Frames are switched by an ordinary unmanaged layer-2 switch, which is what an embedded system’s backplane usually is.

TDN is not an internet stack. It does not replace one and it does not compete with one. A QSOE machine that must speak to the wider world will do it over TCP/IP like everything else; TDN is for the machines that are, logically, *one system* that happens to be spread over several boards.

1.2 The names

The vocabulary is borrowed from amateur radio, and it is not decoration — it is more accurate than the computing words would be.

- **QSP** is the protocol, and the name of the program that speaks it. In the international Q-code, *QSP* means “will you relay to ...?” — which is precisely this component’s job. It also continues the wordplay QSOE itself is named for: a *QSO* is a contact between two stations.
- A machine on the net is a **station**.
- A station’s name is its **call sign**.
- The set of stations working together is a **net**. That is the radio term for a group of stations keeping a schedule together, and it is also, conveniently, where they live in the pathname space: `/net`.

You will meet one more word. A **fleet** is a numbered net: two independent nets may share one switch without seeing each other, and the fleet id is what tells them apart.

1.3 What it looks like

Here is a complete session on a two-station net — a SpacemiT K3 (call sign **k3**) and a SiFive Unmatched (call sign **unmatched**), plugged into the same unmanaged switch. Nothing was configured beyond a shared roster file; neither machine was told the other’s address.

```
[/home/user]$ ls /net
k3  unmatched

[/home/user]$ cat /net/unmatched/sys/osname
QSOE/L

[/home/user]$ cat /net/unmatched/proc/1/info
```

The second command reads a file on the other machine. It is not a copy, a mount, or a cache: the **read** became a message, the message crossed the wire, and the resource server on the Unmatched answered it exactly as it answers a local one. The third does the same to that machine’s process table.

And note what the two stations are. One is running QSOE/N on the Skimmer microkernel; the other is running QSOE/L on seL4. They are talking to each other, and neither the protocol nor the manager that speaks it contains a line of kernel-specific code — **qsp** is an ordinary **quser/** program, built once and run unmodified on both. That is the point of QSOE’s two-kernel bet, demonstrated on the wire.

1.4 What TDN carries, and what it does not

TDN carries QSOE’s own message passing: **open**, **read**, **write**, **close**, **stat**, directory listings, **qsoe_control**, and an application’s own private messages on a connection it opened itself. If it is a message to a resource server, it crosses.

It does not carry:

- **Memory.** There is no distributed shared memory and none is planned. A pointer means nothing on another station.
- **Processes.** There is no remote **posix_spawn** in 0.2. (QNX’s Qnet has one; §6.10 explains why we are in no hurry to copy it.)
- **Real time.** A message that crosses a wire has crossed a wire. Priority is carried and honored (§6.5), but the latency of a switch is not the latency of an IPC, and no protocol can make it so. Design accordingly: **semantics cross the net; timing does not.**

1.5 Clean-room, and why that matters here

QNX has a transparent-networking product called Qnet, and it is the direct inspiration for this one. QSP is not a reimplementation of it. It is a different protocol, in a different frame format, with different addressing and a different trust model, written from scratch under Apache-2.0. Where the two agree it is because the problem forced it; where they differ, this guide says so and says why — there are four such places and each is a deliberate correction, not an accident of independence.

- Station ids are **globally unique**, not node-local, which deletes an entire translation layer (§2.1).
- Transport parameters are **per peer**, never global (§6.7).
- Traffic is **prioritized**, so a bulk transfer cannot head-of-line-block an urgent message (§6.6).
- Remote credentials are **not taken on trust** (§6.10).

Chapter 2

Stations, Call Signs, and the Net

Before configuring anything it is worth being precise about the four things a net is made of, because each one is a deliberate choice and two of them differ from how QNX does it.

2.1 Station ids are assigned by hand

Every station on a net carries a number: its **station id**. Ids are assigned by a person, written down, and never negotiated. Station 1 means the same machine on every station in the net — including on stations that have never heard from it.

This is the opposite of the usual instinct, which is to have machines discover their own identity, and it is worth defending. A net of this kind is a piece of *equipment*, not a population. On the QSOE console the numbers are printed on the front panel beside the switch, so a station id is a fact about the hardware in the same way that a slot number is. Making it emergent would mean the number could change between boots, and a number that changes cannot be silk-screened onto a panel or written in a service manual.

Two ids are reserved: **0** means “the local station” (this is `ND_LOCAL_NODE`, the value that has always meant *here* in QSOE’s message-passing calls), and the all-ones value is the broadcast station used by discovery. Everything from 1 upward is yours.

The divergence from QNX. Qnet’s node descriptors are *node-local*: the number that means “that machine over there” is different on every machine, so every descriptor that crosses a boundary has to be translated, `netmgr_remote_nd()` exists to do it, and `struct _msg_info` carries two node fields where one would do. QSP’s ids are globally unique, which deletes that entire layer. `struct _msg_info` keeps its single `nd` field and there is no translation call, because there is nothing to translate.

2.2 Call signs

A station’s **call sign** is its name: at most 15 characters, printable, no slashes and no dots, because it becomes a directory name under `/net`. The call signs on the reference net are `k3`, `unmatched`, `vf2`, and `gemu`.

A call sign is not authoritative on the wire. Each station announces its own, but **a call sign written in the configuration is never overwritten by what a station says about itself**. The reason is direct: if the wire could rename a station, a typo — or a machine booted with the wrong argument — could make `/net/unmatched` point somewhere else, and every script naming that path would quietly follow it. The file is the authority. The wire fills in only what the file cannot know, which is the hardware address and whether the station is answering right now.

2.3 The roster, and knowing whether two copies agree

The **roster** is the list of stations that belong to the net. It is a *replicated* object, not a service: every station holds the whole thing and resolves names locally. There is no directory server, no election, and no station that is the authority — which is what lets a station beyond the local segment (§3.7) be known at all, since it never announces itself here. It is known because it is written down, exactly as a dial-up network’s node list carried machines nobody could hear.

What a replicated object needs is a way to tell whether two copies agree. That is the **roster serial**: a number in the configuration file, bumped by hand whenever the peer list changes, that every station includes in every announcement it sends. A station running from a stale copy is therefore noticed without anyone being asked — **qspinfo** marks the disagreeing entries (§5.1). A serial of 0 means “unversioned”, which is the honest answer for a net whose roster nobody has bothered to version yet.

2.4 Configured stations and discovered ones

A station’s entry in the local roster came from one of two places, and the difference shows.

Configured. Named by a **peer** line in `/etc/qsp.conf`. Such a station is listed **whether or not it is answering**, and `/net/callsign` resolves for it from the moment **qsp** starts — before that machine has been switched on. Opening something beneath it then fails with “that station is not answering”, which is the truth, rather than “no such path”, which would send you looking in the wrong place. The operator declared it part of the net; its absence is information.

Discovered. Heard announcing itself, but not in the file. Such a station appears when it speaks and is forgotten when it goes quiet.

2.5 /net, the namespace of stations

Every station the local roster knows about appears as a directory under `/net`, named by its call sign. `ls /net` is therefore the answer to “who is out there”:

```
[/home/user]$ ls /net
k3  unmatched  vf2
```

The local station is there too, under its own call sign. That is not decoration: it means a script can name `/net/$NODE/dev/ser1` and run unchanged on every machine in the net, and it means the loopback case — reaching yourself through `/net` — works and is tested rather than being a special case nobody exercises.

stat /net reports the number of stations known as the directory’s size, so a script can check that the net is the size it should be without parsing anything.

2.6 Node descriptors

Some QSOE calls address a station numerically rather than by path. The number is a **node descriptor** (**nd**), and it is a superset of the station id:

$$\text{nd} = (\text{path_selector} \ll 16) \mid \text{station_id}$$

The *path selector* chooses among several links to the same station. The VisionFive 2 has two Ethernet MACs; an application may want a real-time stream pinned to one of them and everything else on the other. The selector is deliberately excluded from identity comparison, so

two descriptors for the same station compare equal however they were reached — QNX does this too, and it is right.

A node descriptor is obtained, never constructed. The encoding above is documented so that a number in a debugger can be read, not so that programs can build one. Obtain descriptors with `qsp_strtond()` or by resolving a path under `/net`, and they will keep working if the encoding ever changes — which it may, since high station ids are a plausible place to put per-process handles later.

```
#include <qsoe/qsp.h>

uint32_t nd = qsp_strtond("unmatched"); /* 0 on failure */
char      sign[16];
qsp_ndtostr(nd, sign, sizeof sign);      /* back to a name */
```

Both calls answer from the manager's own roster, reached with an ordinary `qsoe_control` on `/net`. That is deliberate: a second copy of the net's membership — a cached table, another file — is a copy that can disagree with the one actually routing frames, and a name that resolves differently depending on who asks is worse than a name that does not resolve at all. There is one roster on a station, and these read it.

Failures are distinguished:

<code>ENETDOWN</code>	No manager is running here — this machine is not on a net at all.
<code>EHOSTUNREACH</code>	The roster lists no such station.

A station that is listed but merely not answering *resolves fine*. Being unreachable is a property of the moment, not of the name; the caller finds out when it tries to connect.

Chapter 3

Setting Up a Net

A working net needs three things: a wire, a driver that owns the interface on each machine, and a roster that every station agrees on. This chapter covers all three, and ends with a two-station net brought up from nothing.

3.1 What the hardware has to be

An ordinary **unmanaged gigabit switch**, and one Ethernet port per station. That is the whole requirement. There is no router, no DHCP server, no addressing to plan, and nothing to configure on the switch — QSP frames are switched by hardware address like any other Ethernet traffic, and the protocol’s own addressing rides inside them.

QSP shares the wire politely. Its frames carry EtherType 0x88B5, which is IEEE 802’s *Local Experimental EtherType 1* — the value the standard sets aside for exactly this case, a private protocol on a local network. No registration is needed and no assigned protocol can collide with it. Other traffic on the same switch is ignored, and a ‘Q’ ‘S’ ‘P’ magic in the header tells our frames apart from anyone else’s experiment.

A registered EtherType costs US\$4,170 from the IEEE and would buy nothing on a closed switch. The experimental value is not a shortcut; it is the correct choice for this use.

3.2 The interface driver

QSP does not talk to hardware. It opens `/dev/net/en0` and sends frames through it, so one of the network drivers has to be running first. Which one depends on the board:

Board	Controller	Driver
SiFive Unmatched (FU740)	Cadence GEM	<code>/sbin/devn-gem</code>
SpacemiT K3 (K1)	DesignWare EQoS	<code>/sbin/devn-dwmac</code>
StarFive VisionFive 2	DesignWare EQoS	<code>/sbin/devn-dwmac</code>
QEMU virt	virtio-net	<code>/usr/sbin/devn-virtio</code>

Each registers `/dev/net/en0` and then detaches, so the command returns as soon as the interface exists — there is nothing to wait for. A driver that finds no controller returns nonzero *before* detaching, which is what lets a boot script tell “no hardware” from “started fine”.

3.3 The roster file

`/etc/qsp.conf` is the net’s roster. One directive per line; `#` begins a comment.

<code>station <i>id</i> <i>callsign</i></code>	who <i>we</i> are
<code>fleet <i>id</i></code>	which net this is (default 1)
<code>roster <i>serial</i></code>	which edition of this file it is
<code>peer <i>id</i> <i>callsign</i> [<i>mac</i>]</code>	a station we expect to see
<code>via <i>id</i> <i>gateway-id</i></code>	reach <i>id</i> through <i>gateway-id</i>

A **peer** line needs no hardware address: it is learned from that station's first announcement. Giving one anyway lets the net work with broadcast discovery turned off.

Here is the reference net's file in full:

```
fleet 1
roster 2

peer 1 k3
peer 2 unmatched
peer 3 vf2
peer 9 qemu
```

Every station in the net gets **this same file**, byte for byte. That is not a convenience — it is what makes the roster a replicated object rather than four opinions. Bump the **roster** serial whenever the peer list changes, so that a machine still running the old copy is visible (§2.3).

3.4 Who *we* are: ask the board

Notice what the file above does not contain: a **station** line.

One QSOE system image boots every machine in the net. A **station** line would therefore have to be edited per board — and any board that got the unedited file would quietly become station 1, which is the worst possible way to find out. So the station's own identity comes from the one thing that genuinely differs between the machines: the hardware.

`/usr/conf/sysinit/10-net.sh` reads `/sys/board`, which is the root device tree's **compatible** string verbatim from firmware, and maps it to an id, a call sign, and a driver:

```
read -r QSOE_BOARD < /sys/board

case "$QSOE_BOARD" in
    *spacemit*)      NODE=k3          NODE_ID=1  NETDRV=/sbin/devn-dwmac      ;;
    *sifive*)        NODE=unmatched  NODE_ID=2  NETDRV=/sbin/devn-gem       ;;
    *starfive*)      NODE=vf2         NODE_ID=3  NETDRV=/sbin/devn-dwmac      ;;
    *riscv-virtio*)  NODE=qemu        NODE_ID=9  NETDRV=/usr/sbin/devn-virtio ;;
    *)               NODE=            NODE_ID=   NETDRV=                ;;
esac
```

The match is on the **vendor prefix**, not the whole string. A board's compatible carries a revision suffix (**sifive**, **hifive-unmatched-a00**) that can change under you; the vendor will not.

A board that falls through the **case** gets no id, and says so:

```
sysinit: board "acme,widget" is not in the roster -- no station id,
        so this machine stays off the net.
```

That is the intended behavior for an unknown board. A machine with no identity must not join a net, because the only identity it could invent is one that already belongs to somebody.

3.5 Running the manager

The boot script starts the driver, and starts **qsp** only if the driver found an interface:

```

if "$NETDRV"; then
    /usr/sbin/qsp -s "$NODE_ID" -n "$NODE"
else
    echo "sysinit: $NETDRV found no interface -- $NODE stays off the net."
fi

```

qsp takes five options, and a **command-line value outranks the file**:

-c <i>path</i>	configuration file (default /etc/qsp.conf)
-s <i>id</i>	station id
-n <i>sign</i>	call sign
-i <i>path</i>	interface (default /dev/net/en0)
-r <i>serial</i>	roster edition

On success it prints one line and detaches:

```

qsp: station 2 "unmatched" on /dev/net/en0
      (70:b3:d5:92:f1:0c), net 1, 1452 B/frame

```

The frame size is taken from what the interface says it will carry whole, never from the Ethernet maximum. A frame built larger than the driver's MTU is split on its way out and arrives as two things that are not frames at all.

Starting without an id is refused. A station that cannot be addressed must not answer, so qsp says why and exits rather than sitting silently on the wire:

```

qsp: no station id configured (need a 'station <id> <callsign>' line
      in /etc/qsp.conf); refusing to join the net

```

3.6 Bringing up a two-station net

Everything above, in order, on two machines. Cable both to the switch and power them on; if the roster file is already in the image, the boot scripts do all of this and you need only check the result.

```

# ---- on the K3 ----
[/]# /sbin/devn-dwmac
[/]# /usr/sbin/qsp -s 1 -n k3
qsp: station 1 "k3" on /dev/net/en0 (b2:2c:9e:04:11:87), net 1, 1452 B/frame

# ---- on the Unmatched ----
[/]# /sbin/devn-gem
[/]# /usr/sbin/qsp -s 2 -n unmatched
qsp: station 2 "unmatched" on /dev/net/en0 (70:b3:d5:92:f1:0c), net 1, 1452 B/frame

```

Each station announces itself immediately rather than waiting for its first timer, so the other side learns of it at once. Within a second or so, on either machine:

```

[/]# ls /net
k3  unmatched

```

That is a net. Chapter 4 is what to do with it.

3.7 Stations beyond this switch

A station that is not on this Ethernet segment cannot be discovered, and cannot be reached by hardware address. It is reached through another station that relays for it:

```
peer 16 brazil
via 16 2
```

Frames for station 16 are then addressed to *station 2's* hardware address while still carrying 16 in the QSP header, and station 2 forwards them. That one indirection is the entire mechanism, and it is why QSP needs no routing layer: **forwarding is decided on the destination station id, never on a MAC address.**

The gateway itself is outside this design and need not run QSOE; it is a program that unwraps QSP frames and puts them somewhere else — an encrypted IPv6 tunnel, for instance. Two things make that possible and both are already here: the **via** indirection above, and per-peer transport parameters (§6.7), because a peer 50 microseconds away and a peer 200 milliseconds away cannot share a retransmit timer.

Remember §1.4: semantics cross the planet, real time does not. A remote **read** through a tunnel still returns the right bytes and the right **errno**. It does not still return them in 40 microseconds.

Chapter 4

Using the Net

There is no networking API in this chapter, because there is no networking API. Using the net means using the pathname space.

4.1 The path form

Any path on any station is reached by prefixing it with that station's directory under `/net`:

/net/callsign/path-on-that-station

So `/net/unmatched/dev/ser1` is `/dev/ser1` on the station whose call sign is `unmatched`. The prefix is stripped by that station's manager; what the far machine opens is exactly the remainder.

Everything you would do to a local path works:

```
[/home/user]$ ls /net/k3/dev                # its device directory
[/home/user]$ cat /net/k3/sys/board          # what board it is
[/home/user]$ cat /net/k3/proc/1/info        # its taskman
[/home/user]$ cp /net/k3/etc/qsp.conf /tmp/theirs # read a file across
[/home/user]$ echo hello > /net/k3/dev/ser1  # write to its serial port
```

A bare `/net/callsign` with nothing after it is that station's root directory.

Your own station is in there too. `/net/k3/...` on the K3 resolves locally and never reaches the wire. This is worth relying on: a script can name `/net/$NODE/dev/ser1` and be correct on every machine in the net, with no special case for “the local one”.

4.2 What a remote open actually does

Understanding this takes ten lines and explains most of the behavior you will see.

When you open `/net/unmatched/dev/ser1`:

1. The path manager resolves `/net/unmatched` to the local `qsp`, which is registered there. Longest-prefix resolution sends the `/net` directory itself to one part of `qsp` and everything beneath a station to the relay. **This is decided once, at open, and never again per message.**
2. `qsp` strips `/net/unmatched`, and sends a `CONNECT` frame carrying the remainder — `/dev/ser1` — your open flags, mode, uid and gid.
3. The `qsp` on the Unmatched performs an ordinary local `open("/dev/ser1")` and keeps the descriptor.

4. Its status comes back and becomes the status of your `open`.

Two consequences follow directly.

The remote server needs no awareness of QSP whatsoever. What `devc-ser8250` on the Unmatched sees is a local client doing a local open, followed by local messages. Any resource server written for QSOE is network-transparent the day it is written, and there is nothing to add to it.

A circuit is opened by *path*, never by a bare channel id. That is what lets the two kernels interoperate in one net. QSOE/N's channel ids are node-global; QSOE/L's are per-process. A number would mean different things on the two machines — a name means the same thing on both.

4.3 Everything after the open is opaque

The relay looks at exactly two message types: the open that names the resource, and the close that releases it. **Everything else is forwarded without being read.**

This matters if you have written your own resource server with its own private message protocol. It crosses the net unchanged and un-decoded; the relay does not know what your messages mean and does not need to. Your reply comes back byte for byte, with the status your server returned.

`dup` is the one further case worth naming, and it does the right thing: a `dup` crossing the wire arrives as a `dup`. The far station duplicates its own descriptor, so whatever per-open state the server there keeps comes along. Reopening the path instead would have produced a descriptor merely pointing at the same resource, which is not what `dup` means.

4.4 Errors, and what each one tells you

The failures are distinguished on purpose — each points at a different place to look.

ENETDOWN	There is no <code>/net</code> on this machine: no manager is running here. Look at your own station, not the other one.
EHOSTUNREACH	The roster lists no such call sign. A configuration mistake — a name that was never declared.
EHOSTDOWN	The station is in the roster but is not answering. A cable, a switch port, or a machine that is off.
ETIMEDOUT	The exchange was in flight and the station went away underneath it.
EBUSY	This station has no free circuit or transaction slot (§4.7).
ENOENT, etc.	The far station answered, and this is <i>its</i> <code>errno</code> — the path does not exist over there. A remote error is reported as itself.

The distinction between `EHOSTUNREACH` and `EHOSTDOWN` is the one worth internalizing: one of them is a typo in a file and the other is a cable.

This is also why a *configured* station resolves while it is switched off (§2.4). Had it simply not appeared under `/net`, you would get “no such file or directory” and spend the next ten minutes looking for a spelling mistake that is not there.

4.5 Addressing a station from a program

Most programs need nothing beyond the path form — that is the point. When a program does need to name a station numerically, it obtains a node descriptor (§2.6):

```
#include <qsoe/qsp.h>
#include <sys/qsoe.h>

uint32_t nd = qsp_strtond("unmatched");
if (nd == 0) {
    /* qsoe_errno is ENETDOWN (no manager here) or EHOSTUNREACH
     * (this net has no such station). */
}
```

Planned content. `ConnectAttach` with a non-zero `nd` is the second route to a remote resource, and 0.2 does not yet have it: today the kernel still refuses a non-zero node descriptor and the pathname form above is the supported way. When it lands it will be handled entirely in the shared libc — it will open an ordinary *local* connection to `qsp` and bind it, so that the returned `coid` is a real local `coid` and `MsgSend` is never intercepted. The local IPC fast path will pay nothing for the feature’s existence, which is the constraint the design is built around.

4.6 What the far station sees

The originating uid and gid *are* carried in the `CONNECT` frame. In 0.2 they are not yet acted upon: the far station performs its `open` with the manager’s own credentials, so a remote open is as privileged as `qsp` is there.

Say plainly what that means. **A net is currently a trust domain.** Any machine that can put frames on the switch can open anything on any station in the net that carries its fleet id. For a closed piece of equipment whose boards are all yours — which is what 0.2 targets — that is an acceptable and clearly bounded position. For anything else it is not, and §6.10 says what the protocol already reserves in order to fix it, and why QSOE will not fix it the way Qnet did.

Planned content. Per-peer credential policy: a peer entry gains a rule for what to do with the uid and gid it claims — honor them for a peer on the local segment, map them all onto one unprivileged uid for a peer reached through a gateway. That, together with the frame header’s reserved authentication tag, is what turns the carried credentials into enforced ones.

4.7 Limits you can hit today

These are properties of the current implementation, not of the protocol, and each is a compile-time constant in the manager:

Stations in the roster	32	A net this size is a console, not a data centre.
Circuits (each direction)	32	Open remote descriptors.
Concurrent transactions out	8	In-flight <code>MsgSends</code> to other stations.
Concurrent transactions in	4	Being served for other stations.
Relay threads	4	Local clients carried at once.
Bytes in one transaction	8192	Either direction.

The message-size limit deserves a word, because it sounds tighter than it is. Every POSIX operation fits with room to spare: the resource-manager protocol caps one `read` or `write` payload

at 896 bytes, so a request or a reply crosses in a single frame. The 8 KiB figure exists for an application sending its own large message on a remote connection, and it is a property of *this implementation* — the wire header’s offset and total fields describe a message far larger. What holds it here is that the far side reassembles into a fixed buffer.

Exhausting a table gives **EBUSY** rather than blocking. That is deliberate: a fixed table with an honest refusal is a system whose worst case you can compute, and the alternative is a queue with no bound on how long you wait in it.

Chapter 5

Diagnosing a Net

A distributed system's first diagnostic is not “does it work” but “do the machines agree”. Two tools answer that, at two different layers.

5.1 qspinfo — what this station believes

Run with no argument, `qspinfo` prints who we are and the roster as we hold it:

[/home/user]\$ qspinfo							
station	2	"unmatched"	net	1	roster	2	
hardware	70:b3:d5:92:f1:0c						
uptime	412.338	s					
frames	rx 8114 (dropped 0, other nets 0)	tx 8109 (errors 0)					
announce	sent 206, heard 205						
id	call sign	state	how	hardware	roster	mtu	
1	k3	up	listed	b2:2c:9e:04:11:87	2	1452	
2	unmatched	self	listed	70:b3:d5:92:f1:0c	2	1452	
3	vf2	down	listed	00:00:00:00:00:00	0	0	
9	qemu	down	listed	00:00:00:00:00:00	0	0	

Read it column by column:

state **self** is this machine; **up** means the station answered within the liveness timeout; **down** means it did not. A **down** station is still listed because it was configured (§2.4).

how **listed** — named in `/etc/qsp.conf`; **heard** — discovered on the wire and not in the file; **via** *n* — reached through gateway station *n*.

hardware Learned from that station's first announcement. All zeroes means we have never heard from it.

roster The roster edition *that station announced*.

mtu The largest payload that station said it will accept. Zero until it has spoken.

The line that matters most. A station running from a different edition of the roster is flagged:

1	k3	up	listed	b2:2c:9e:04:11:87	1	1452	<- differs
---	----	----	--------	-------------------	---	------	------------

That is the K3 answering happily while working from roster 1 when we are on roster 2 — somebody updated the file on one machine and not the other. It is worth flagging precisely because everything else still works: the two stations talk, the ping succeeds, and the disagreement

shows up much later as a station that one machine can name and the other cannot. Every other difference between two stations is expected; this one is not.

Resolving a name. With a call sign as argument, `qspinfo` makes the round trip an application makes before it can address a station at all:

```
[/home/user]$ qspinfo k3
k3 -> nd 0x00000001 (station 1, path 0) -> k3
```

If that fails, no program on this machine can reach that station, and the `errno` says which half is wrong (§2.6).

5.2 netpoke — is there a wire at all

`qspinfo` answers questions about the net. When the answer is “no stations at all”, the question moves down a layer: does this interface actually put frames on a cable, and does anything come back? That is **netpoke**, which speaks no protocol whatsoever — it puts a recognizable payload inside our `EtherType` and prints what arrives.

<code>netpoke stats</code>	the interface’s own counters
<code>netpoke rx [sec]</code>	listen and print frames
<code>netpoke tx text [count]</code>	broadcast <i>text</i>
<code>netpoke ping text</code>	broadcast once, then listen for replies
<code>netpoke echo [sec]</code>	reply to every request, unicast back

The useful pairing is `echo` on one machine and `ping` on the other:

```
# ---- on the K3 ----
[/]# netpoke echo 30

# ---- on the Unmatched ----
[/]# netpoke ping hello
```

A reply proves the entire physical path — driver, PHY, cable, switch port, and the same in reverse — with nothing above it involved. No reply, with `netpoke stats` showing frames transmitted and none received, is a wiring or a PHY problem and not a QSP problem.

5.3 Common failures, in the order to check them

ls /net says no such file or directory. No manager is running on this machine. Look at the boot output: either the driver found no controller, or `qsp` refused to start because the board was not in `10-net.sh`’s case (§3.4). Both say so in as many words.

/net exists but is empty. The manager is running and has heard nothing, and nothing was configured either. Check that `/etc/qsp.conf` is present — `qsp` prints `no /etc/qsp.conf - continuing unconfigured` when it is not, and an unconfigured station lists only whom it hears.

Stations are listed but all down. They are configured and not answering. Check `qspinfo`’s frame counters: `tx` climbing with `rx` at zero is a receive-side problem; both at zero is an interface that is not running. Drop to `netpoke` (§5.2).

Everything is up but the wrong machines are talking. Check the `net` number in `qspinfo`'s first line. Two nets sharing one switch do not see each other, by design — frames carrying a foreign fleet id are counted as `other nets` and discarded. A non-zero `other nets` counter with an otherwise silent net means your fleet ids disagree.

One station can name something the others cannot. Roster skew. Look for `<- differs` (§5.1), fix the file, and remember to bump the serial — an unbumped serial is exactly the case this check cannot catch.

A remote open fails. Read the `errno` against the table in §4.4 before anything else. It distinguishes “no such station” from “that station is off” from “the path does not exist over there”, and each sends you to a different machine.

A station announces itself and vanishes repeatedly. Look at `announce sent` versus `heard`. A station whose announcements are heard but whose pings are not is usually two stations sharing an id — both answer, each thinks the other is itself, and the roster flickers. Ids are assigned by hand precisely so that this is a thing you can check on paper.

Chapter 6

How QSP Works

Nothing in this chapter is needed to use the net. It is here because the design has opinions, and because knowing where the bytes go is what lets you predict how a system built on it will behave.

6.1 Three layers

Frame. EtherType 0x88B5, a 48-byte header after Ethernet’s 14. Magic and version, fleet id, 32-bit source and destination station, verb, flags, header length, circuit and transaction ids, sequence and acknowledgement, the fragment’s offset and the message’s total, payload length, and a reserved authentication tag.

Link. Per-peer sequencing, liveness, and — when it is finished — retransmission and duplicate suppression. There is no congestion control and none is planned: this is a switch, not the Internet.

Transaction. Circuits and the exchanges carried on them, which is where QSOE’s own message passing appears on the wire.

The frame header is fixed and flat: no option TLVs, no variable-length fields, every field naturally aligned, little-endian by declaration rather than negotiation. That is not minimalism for its own sake. One station on the target net is an RP2350 microcontroller, and the header must be parseable by a few hundred bytes of code with no dynamic allocation. A protocol that can be implemented on the smallest station in the net can be implemented anywhere.

The one concession to the future is `hdr_len`, which lets a later version grow the header without a flag day, and the `tag` field, reserved now so that authenticating a station later moves no field.

6.2 Discovery and liveness

A station announces itself on joining, then every two seconds. A peer that has not been heard from is probed once a second, and is declared gone after five seconds of silence. An announcement carries the station’s call sign, its MTU, its capabilities, its uptime — which lets a peer notice a restart — and its roster serial (§2.3).

Discovery frames are broadcast; everything else is unicast. A frame whose source station is our own is dropped on sight, because a shared segment (QEMU’s in particular) delivers a broadcast back to its sender, and a station answering itself is how you get an echo storm.

6.3 Circuits

A **circuit** binds one client connection here to one open resource there. It is opened by `CONNECT` naming a path, and identified by the pair (initiating station, circuit id) — the id is allocated by whoever opened it, so no negotiation is needed and the two ends can never race to name it.

Each station keeps two tables, because it is both ends of the relationship: circuits it opened for its own clients, and circuits opened *on* it, each holding the descriptor it opened on the far client's behalf.

A client circuit is keyed by the **server-side connection id** that the kernel stamps on every message from that connection. This is the detail that makes the whole thing cheap: the connection itself is the circuit handle, so nothing needs a per-message lookup and no per-message header rides along. The relayed bytes are the client's bytes, verbatim.

6.4 Transactions

One transaction is one `MsgSend` and its reply. Its bytes travel as a `SEND` fragment and as many `SEND_CONT` fragments as are needed, answered by `REPLY` and `REPLY_CONT`; `offset` and `total` place each fragment, and a `MORE` flag marks every fragment but the last.

The first fragment in each direction carries a small preamble — the client's reply capacity, the originating thread's priority — ahead of the message bytes. Continuation fragments carry message bytes only, so the preamble costs nothing per fragment.

A request is never held whole on the sending station: fragments are pulled from the client's own buffer as they go out.

One round trip, not many. This is the second correction to Qnet. In QNX, a server that issues `MsgRead` against a remote client costs a round trip *per read*, so a server written naturally for local clients becomes latency-bound the moment one of them is remote. QSP reassembles the message on the far station and issues **one ordinary local `MsgSend`** to the target server. A remote server therefore behaves exactly like a local one, and the cost lands where it can be bounded: one reassembly buffer per in-flight inbound transaction.

No kernel virtual threads. Qnet fabricates a non-schedulable placeholder thread in the kernel for every in-flight remote client, and turns every server-side `MsgReply`, `MsgRead`, and `MsgWrite` into a pulse upcall to the net manager. QSOE instead makes the far-side `qsp` an ordinary local client of the target server. The kernel is not involved at all — neither kernel is, which is why one binary serves both. The price is that a worker pool bounds concurrency (§4.7), which is a price you can compute.

6.5 Priority crosses the wire

The `SEND` preamble carries the originating thread's priority, and the far-side worker *sets its own priority to match* for the duration of the exchange, restoring it afterward. A high-priority client on another station is therefore not served behind a low-priority one — priority inheritance, extended across the net.

The priority is clamped to the local range. Stations need not have the same scheduling range, and the RP2350 will not.

6.6 Traffic classes

Each peer has more than one traffic class — URGENT and BULK — each with its own sequence space. A transaction whose message exceeds one frame goes as BULK; everything else, and all control traffic, goes as URGENT.

This is the third correction to Qnet, and the reason is arithmetic. At gigabit speed a 4 MiB bulk transfer occupies the wire for roughly 35 milliseconds. If there is a single ordered stream per peer, an urgent 64-byte message that arrives one frame into that transfer waits for all of it. For a fabric carrying audio and control traffic, tens of milliseconds of head-of-line blocking is the difference between a demonstration and a real-time system. Ordering is only required *within* a circuit, which is all the Send/Receive/Reply contract asks for, so separating the classes costs nothing that anybody promised.

Planned content. The classes exist, are carried, and have separate sequence spaces; what 0.2 does not yet have is the drain-highest-first transmit scheduler that makes the guarantee real under load.

6.7 Everything is per peer

MTU, retransmit timer, window, heartbeat interval: all of these are properties of a *peer*, never of the station.

This is the fourth correction to Qnet, which made `mtu_en` a global that had to match on every node. Two things break under that rule. A net of mixed hardware has peers with genuinely different frame sizes. And a peer reached through a tunnel is both slower and smaller than one across the switch: 50 microseconds and 200 milliseconds cannot share a timer, and encapsulation forces smaller frames at the source. Making these per-peer costs nothing today and is what keeps §3.7 possible.

6.8 Why the manager does not use the resource-server framework

QSOE has a resource-server framework whose job is to decode an incoming request into a typed method call. That is exactly what a transparent relay must never do.

So `qsp` hand-rolls its receive loop for the relay channel. It looks at two message types — the open that names the resource and the close that releases it — and forwards everything else without understanding it. The serial driver `devc-ser8250` hand-rolls its loop for the same reason. The `/net` directory itself *is* served by the framework, because a directory is an ordinary typed thing; the split is decided once by longest-prefix path resolution and costs a client nothing.

6.9 The threads

Four kinds, and the split is forced rather than chosen:

The wire thread owns the interface and never blocks. It drains received frames, runs discovery, and supervises in-flight transactions — which is where a client parked on a station that has gone away is released.

The dispatch thread serves `/net` itself through the framework.

Relay threads each carry one local client's request out to another station and park for its duration.

Inbound workers each serve one thing another station has asked of us, at the priority that request carried.

The wire thread never touches a client's message, and that is a kernel rule rather than a style preference: `MsgReadv` and `MsgWritev` act on the message the *calling* thread received. So the wire thread only fills in reply bytes and says when they are all there; the relay thread that took the request does every transfer and the final reply.

6.10 What QSOE refuses to inherit

Qnet's `NetCred` lets a net manager write arbitrary uid and gid values onto a connection with **no verification whatsoever**, and the kernel then uses them for real access decisions — including remote process creation. The practical consequence is that any machine able to speak Qnet can claim uid 0.

QSOE will not do this. The protocol reserves, from its first frame, the pieces needed to do it properly: a per-frame authentication tag with a flag bit that says it is present, and a per-peer policy for what a station's claimed credentials are worth. With those, a peer on the local segment can be trusted to name its own users while a peer reached through a gateway has every uid mapped onto one unprivileged id, and a station is authenticated end to end regardless of what relayed its frames.

§4.6 states where 0.2 actually stands, which is that the credentials are carried and not yet enforced. The distinction between that and Qnet's position is not the amount of security implemented — today it is comparable — but that one design has somewhere to put the fix.

Chapter 7

The State of TDN in 0.2

TDN is the flagship feature of QSOE 0.2, and 0.2 is a release candidate series, not a finished product. This chapter is the honest inventory: what you can rely on, what you cannot, and what each missing piece will change when it lands.

7.1 What works

- **Discovery and liveness.** Stations find each other on an unmanaged switch with no configuration beyond a shared roster, and notice within five seconds when one goes away.
- **The `/net` namespace.** Every known station appears as a directory, configured stations included while they are down, the local station included as itself.
- **Remote open, and everything after it.** `open`, `read`, `write`, `close`, `stat`, directory listings, `qsoe_control`, `dup`, and an application's own private messages, all crossing unchanged, with the far station's `errno` reported as itself.
- **Both kernels, one binary.** `qsp` is an ordinary `quser/` program. A QSOE/N station and a QSOE/L station on one switch are talking through the same code.
- **Priority inheritance across the wire.**
- **Node descriptors** obtained by name, from the manager's own roster.
- **Roster-skew detection**, so two machines working from different editions of the net's membership are visible rather than inferred.
- **The gateway indirection.** `via` resolves to the gateway's hardware address while the frame keeps the destination's station id, which is the whole mechanism an off-segment relay needs.

7.2 What does not, yet

Reliable transport. This is the significant one. The frame header carries a sequence number and a cumulative acknowledgement, sequence numbers are assigned per peer and per class, and *nothing acts on them*. There is no retransmission and no duplicate suppression.

On a quiet unmanaged switch between two directly-cabled stations, frames do not in practice get lost, which is why the layers above could be built and tested first. But be clear about the failure mode: a lost data frame today does not produce an error — it produces a transaction that never completes, and a client that stays blocked until the far station goes quiet altogether and the supervision releases it with `ETIMEDOUT`. A `CONNECT` has a five-second deadline; a

data transaction deliberately has none, because reading a serial port legitimately blocks until somebody presses a key, and the thing that must end such a wait is the station going away rather than a clock.

The transmit scheduler. Traffic classes have separate sequence spaces but frames are transmitted as they are produced. The head-of-line guarantee of §6.6 is designed in and not yet enforced under load.

ConnectAttach to a remote node descriptor. The kernel still refuses a non-zero `nd`. The pathname form is the supported route (§4.5).

Enforced credentials. Carried, not applied (§4.6). A net is a trust domain in 0.2.

Per-peer transport parameters in the configuration. The manager's peer table has the fields; `/etc/qsp.conf` has no syntax for setting them yet, so every peer currently uses the same timings.

The gateway itself. `via` works. The program that would sit at the far end of it — unwrapping QSP frames into an encrypted IPv6 tunnel — does not exist and is outside this design.

7.3 Sizing a system against this

Three numbers decide whether a design fits.

Concurrency. Four relay threads out and four inbound workers in. A fifth simultaneous remote request is refused with `EBUSY`, not queued. If your design has more than a handful of things talking across the net at once, raise the constants or rethink the shape.

Message size. 8 KiB per transaction, in either direction. Every POSIX operation is far below it; only an application's own large messages approach it.

Latency. A remote message is a round trip across a switch plus a relay thread's scheduling on each side. It is not an IPC and no amount of protocol work will make it one. Put the real-time loop on one station and let the net carry what does not have a deadline.

7.4 The road

In rough order of what would be picked up next:

1. Retransmission, cumulative acknowledgement and duplicate suppression on the sequence fields that are already there and already carried.
2. The priority-aware transmit scheduler, which is what makes the traffic-class design load-bearing.
3. Per-peer transport parameters in `/etc/qsp.conf`.
4. `ConnectAttach` with a node descriptor, handled in the shared `libc` so that the local IPC fast path pays nothing.
5. Per-peer credential policy and the frame authentication tag, which together end the trust-domain assumption.

The 0.2 gate is two stations on the console's front-panel switch — a SpacemiT K3 running QSOE/N and a SiFive Unmatched running QSOE/L — reaching each other's resources by name. The VisionFive 2 as station 3 and the RP2350 as station 4 come after.